



VIA University College

23/10/1944 - Procesrapport

Nikolaj Bræmer Christensen – 354322

Jonathan Cilius Østberg Winther Engell Gøtze – 354500

Victor Bruun Fassbender – 354361

PELC

Antal tegn:

XR Softwareingeniøruddannelsen

4. semester

26. maj 2026

Indhold

1	Indledning	2
2	Gruppearbejde	4
3	Projektinitiering	6
4	Projektudførelse	8
4.1	Oprettelse af projekt	8
4.2	Game Sessions	11
4.3	TeamSelection	13
4.4	VoiceChat	15
4.5	Timer	16
4.6	Class Selection	17
4.7	Speceille Evner	19
4.8	nøgler og kisten	20
4.9	Fange Mechanic	22
4.10	befri fængslet	23
4.11	Win Conditions	24
4.12	map opsætning	25
4.13	AI NPC'er	27
4.14	aflsuttende forbedringer	28
5	Personlige Refleksioner	29
5.1	Jonathan	29
5.2	Victor	30
5.3	Nikolaj	31
6	Refleksion over vejledning	32
7	Konklusion	33

1 Indledning

I anledning af, at Viborg Katedralskole fejrer bygningens 100-års jubilæum, valgte gruppen at udvikle et spil, der både fejrer og hylder skolens historie, arkitektur og kulturelle betydning. Gruppen ønskede at skabe et projekt, som kombinerer moderne teknologi med historisk formidling, og som samtidig kunne give brugeren en interaktiv oplevelse af skolens omgivelser og historie. Motivation for projektet opstod ud fra et ønske om at arbejde med et lokalt og meningsfuldt emne, hvor der kunne skabes en tæt forbindelse mellem det tekniske produkt og den historiske fortælling.

I projektets opstartsfasen arbejdede gruppen med idéudvikling og research omkring skolens historie og arkitektoniske udtryk. Gennem møder med skolens historielærere og gennemgang af historisk materiale blev gruppen introduceret til skolens rolle under Anden Verdenskrig. Her blev det blandt andet beskrevet, hvordan modstandsfolk benyttede skolen til produktion og distribution af illegale aviser under den tyske besættelse. Gruppen fandt denne fortælling særligt interessant, da den både havde historisk betydning og samtidig kunne omsættes til spændende gameplay-elementer et digitalt spil.

På baggrund af dette valgte gruppen at udvikle et online multiplayer-spil, hvor spillerne enten kan spille som modstandsfolk eller som nazister. Modstandsfolkene skal bevæge sig rundt på skolen og udføre forskellige opgaver, såsom at Finde Nøgler som refere til Skolens Logo og Historie eller undgå at blive opdaget, mens den modsatte side forsøger at finde og stoppe dem. Gruppen ønskede gennem dette gameplay at skabe spænding og samarbejde mellem spillerne, samtidig med at spillet tog inspiration fra virkelige historiske begivenheder.

En vigtig del af projektet var at skabe et visuelt realistisk miljø, som kunne give spillerne følelsen af faktisk at befinde sig på skolen. Derfor blev spillet udviklet i Unreal Engine, da gruppen vurderede, at denne engine gav de bedste muligheder for høj grafisk kvalitet, realistisk lysopsætning og detaljerede omgivelser. Gruppen havde desuden et ønske om at lære mere om professionel spiludvikling og arbejde med

værktøjer, der anvendes i den moderne spilindustri.

For at gøre omgivelserne så virkelighedstro som muligt anvendte gruppen 3D-scanninger af skolens bygninger og områder. Dette blev gjort ved hjælp af programmet RealityScan, som gjorde det muligt at omdanne billeder af bygningerne til detaljerede 3D-modeller såkaldt photogrammetry. Disse modeller blev efterfølgende importeret til spillet og viderebearbejdet, så de kunne anvendes i det digitale miljø. Gruppen oplevede, at dette både gav en mere autentisk oplevelse og samtidig gjorde projektet mere teknisk udfordrende, da arbejdet med 3D-scanning og optimering krævede nye kompetencer inden for modellering og performanceoptimering.

Projektforløbet blev organiseret ud fra en Scrum-lignende arbejdsmetode med sprint-baseret udvikling. Gruppen valgte denne agile tilgang for at skabe en fleksibel arbejdsproces, hvor der løbende kunne arbejdes med planlægning, udvikling, test og evaluering. Hvert sprint havde fokus på bestemte funktioner eller dele af spillet, hvilket gjorde det muligt at arbejde mere struktureret og samtidig tilpasse projektet undervejs. Gruppen anvendte blandt andet møder og opgavefordeling, for at sikre overblik over arbejdsprocessen og kommunikationen mellem gruppemedlemmerne.

Igennem projektet oplevede gruppen både tekniske og samarbejds-mæssige udfordringer. Særligt arbejdet med multiplayer-funktioner, 3D-modeller og koordinering mellem forskellige arbejdsområder krævede løbende problemløsning og samarbejde. Samtidig gav projektet gruppen mulighed for at udvikle kompetencer inden for programmering, projektstyring, kommunikation og kreativ problemløsning. Gruppen fik desuden erfaring med, hvordan historiske fortællinger kan integreres i digitale medier for at skabe en engagerende brugeroplevelse.

Procesrapporten vil derfor beskrive projektets udviklingsforløb, arbejdsmetoder og samarbejde samt gruppens refleksioner over læringsudbytte, udfordringer og erfaringer gennem hele projektperioden.

2 Gruppearbejde

Gruppen har som tidligere nævnt arbejdet ud fra en Scrum-lignende proces gennem hele projektforsløbet. I denne struktur har gruppemedlemmerne haft forskellige roller, som både har været baseret på personlige styrker og på deres samarbejdsevner.

Nikolaj har fungeret som gruppens Scrum Master gennem hele projektet. Denne rolle har passet godt til hans strukturelle tilgang til arbejde samt hans erfaring med planlægning og ledelse fra tidligere ophold i forsvaret. Han har haft fokus på at sikre overblik over arbejdsprocessen, holde styr på sprintforløb og understøtte gruppens fremdrift. Derudover har han bidraget til at skabe struktur i arbejdet og sikre, at opgaver blev fordelt og fulgt op på.

Victor har haft rollen som Product Owner. Denne rolle er blevet tildelt ham på baggrund af hans stærke kommunikative evner og hans evne til at formidle idéer klart til både gruppen og eksterne parter. Victor har desuden stået for den primære kommunikation med projektets "kunde", hvilket har været vigtigt for at sikre, at projektets retning og krav blev forstået og opdateret løbende gennem forløbet. Hans rolle har derfor været central i forhold til at afstemme forventninger og sikre, at projektet bevægede sig i den ønskede retning.

Jonathan har fungeret som gruppens analytiske specialist. Han har bidraget med velovervejede analyser, refleksioner og faglige vurderinger i både udviklingen af spillet og i rapportskrivningen. Hans tilgang til arbejdet har i høj grad været præget af en analytisk og struktureret tankegang, som stemmer overens med hans blå DISC-personprofil. Dette har været en styrke i forhold til at sikre kvalitet i gruppens beslutninger og skabe et solidt grundlag for de tekniske og designmæssige valg i projektet.

Gruppen har arbejdet i et Scrum-lignende system, hvor der blandt andet er blevet anvendt Planning Poker til estimering af use cases og tildeling af story points. Dette har hjulpet gruppen med at prioritere opgaver og skabe et bedre overblik over arbejdsbyrden i de enkelte sprint. Projektet har været opdelt i korte sprintforløb, typisk

bestående af intensive arbejdsperioder fra mandag til onsdag, og så fra onsdag til fredag samt et længere “weekend-sprint” fra fredag til mandag. Denne struktur gjorde det muligt at arbejde fokuseret i afgrænsede perioder og løbende evaluere fremdriften.

Hvert sprint blev afsluttet med et retrospektiv og et review-møde. Under retrospektivet blev det gennemførte arbejde gennemgået, og der blev “brændt” story points for færdiggjorte opgaver, hvilket gav gruppen et overblik over fremdriften og arbejdsindsatsen. I review-fasen blev arbejdsprocessen, samarbejdet og de anvendte metoder diskuteret, så gruppen løbende kunne justere og forbedre sin arbejdsgang. Dette har været en vigtig del af den iterative udvikling, da det har givet mulighed for kontinuerlig forbedring af både struktur og samarbejde.

Ved sprint planning blev backloggen gennemgået, og nye use cases blev prioriteret og fordelt mellem gruppemedlemmerne. Dette sikrede en klar forventningsafstemning i begyndelsen af hvert sprint og gav en fælles forståelse af, hvilke mål der skulle opnås.

Gruppen har dog ikke anvendt fuld Scrum-metodologi, da der eksempelvis ikke blev afholdt daglige stand-up møder. Dette var et bevidst valg, da gruppen ønskede en lettere og mere fleksibel tilgang til metoden, som bedre passede til projektets størrelse og arbejdsform. Derudover vurderede gruppen, at den løbende kommunikation internt var tilstrækkelig til at håndtere eventuelle udfordringer uden faste daglige møder. Da sprintene samtidig var relativt korte, oplevede gruppen, at der stadig var en god kontinuerlig forståelse for projektets status uden behov for daglige Scrum-møder.

Samlet set har denne arbejdsform bidraget til en struktureret, men fleksibel proces, hvor roller, planlægning og løbende evaluering har været centrale elementer i gruppens samarbejde.

3 Projektinitiering

Gruppen valgte at holde problemfelt og problemformulering mere åbne for at undgå at definere den endelige løsning for tidligt i projektet. Dette var inspireret af PBL-tilgangen, hvor problemfeltet skal beskrive konteksten uden at skabe tunnelsyn omkring en bestemt løsning eller historisk retning.

Gruppen valgte efterfølgende at tage udgangspunkt i razziaen fra oktober 1943, da denne begivenhed repræsenterede en markant og dramatisk del af skolens historie under besættelsen. Begivenheden gav samtidig mulighed for at kombinere historisk formidling med multiplayer gameplay og samarbejdsmechanikker i Unreal Engine 5. Projektets historiske tema førte naturligt til flere etiske overvejelser. Gruppen diskuterede tidligt i projektet, hvordan historiske roller skulle repræsenteres i spillet. Selvom projektet var inspireret af virkelige hændelser fra besættelsen, var målet ikke at skabe en fuldstændig historisk simulation, men derimod en multiplayeroplevelse inspireret af skolens historie. Der opstod derfor refleksioner omkring balancen mellem historisk autenticitet og spildesign. Gruppen valgte blandt andet, at spillere ikke skulle fremstilles som elever i nazistiske roller. Denne beslutning blev taget for at undgå en u hensigtsmæssig identifikation mellem nuværende elever og historiske aktører fra besættelsen. Samtidig blev flere gameplay-elementer, såsom abilities og movement-mekanikker, designet ud fra etablerede multiplayergenrer fremfor historisk realisme. Dette gjorde det muligt at prioritere et engagerende gameplay-loop, samtidig med at spillet stadig bevarede en historisk inspiration.

Gruppen anvendte primært en Scrum-Lignende metode som overordnet arbejdsproces. Projektet blev opdelt i forskellige sprints, hvor der først blev arbejdet med idéudvikling og planlægning, hvorefter implementation og test blev udført løbende, denne tilgang blev valgt da flere medlemmer i gruppen, tidligere har syntes at et Fuldt Scrum forløb, er for meget Process og det kan være hæmende for at få udviklet noget, omvendt har medlemmerne også oplevet at en meget løser process så som Kanban, er for lidt process også, derfor blev der fundet et kompromis, ved at tage dele fra Scrum, scrum møde, retrospective, rewive, samt sprint planing og planig poker, dog afholdets der ikke dagligscrum meetings, da det vurderes at havde medlemmer brug for hjælp til udviklingen af deres gældende case, så ville de selvstændigt kommunikere det ud

til gruppen, dette blev også valgt da sprint længden var sat til kun 2 dage, for at stadig havde en vis rytme og hastighed i processen. Kommunikationen foregik hovedsageligt gennem Discord, hvor gruppen afholdt online møder cirka hver anden dag. Dette gjorde det muligt at koordinere arbejdsopgaver og følge projektets fremdrift.

4 Projektudførelse

4.1 Oprettelse af projekt

For at komme i gang med projektet oprettede gruppen først selve Unreal Engine-projektet. Projektet blev initialt oprettet som et C++-projekt baseret på Unreal Engine's `FirstPersonTemplate`, da gruppen ønskede, at spillet skulle opleves fra et førstepersonsperspektiv.

Dette valg blev truffet tidligt i processen for at sikre, at gameplay-oplevelsen blev defineret allerede fra starten, hvilket senere fik betydning for designvalg og udviklingen af spilmekanikker.

Efter oprettelsen af projektet blev der opsat et fælles GitHub-repository for at understøtte samarbejdet i gruppen. Dette gjorde det muligt at arbejde parallelt på projektet, samtidig med at der blev skabt en struktureret versionstyring af filer og ændringer.

Brugen af pull requests og code reviews blev implementeret for at sikre kvalitet i koden og for at give mulighed for faglig sparring mellem gruppemedlemmerne. Denne arbejdsform viste sig at være vigtig for at undgå konflikter i koden og for at sikre, at ændringer blev gennemgået og forstået af flere i gruppen, før de blev integreret i hovedprojektet.

Tidligt i udviklingsfasen blev der eksperimenteret med Unreal Engine, særligt i forhold til samspillet mellem `Blueprints` og C++. Gruppen undersøgte blandt andet, hvordan `Blueprints` kunne anvendes til hurtig prototyping af gameplay-elementer, mens C++ blev brugt til mere komplekse og performancekritiske systemer.

Denne kombination viste sig at være central for projektets udvikling, da den gav en balance mellem fleksibilitet og kontrol. Eksempelvis blev gameplay-relaterede funktioner ofte udviklet som `Blueprint`-baserede actors og widgets, mens centrale systemer som netværkslogik, sessionshåndtering og spillerdata blev implementeret i C++-klasser såsom `GameMode`, `GameState`, `PlayerController` og `GameInstance`.

I begyndelsen af projektet bar arbejdet præg af eksperimentering og læring, hvor gruppen brugte tid på at opbygge forståelse for engine'ens struktur og arbejdsmetoder. Dette betød, at fremdriften i starten var relativt langsom, men til gengæld lagde det et solidt fundament for den videre udvikling.

Efterhånden som gruppen fik større teknisk forståelse, blev arbejdsprocessen mere effektiv, og udviklingen af spillets funktioner kunne ske mere struktureret og målrettet. Gruppen begyndte her at arbejde mere systematisk med opdeling af ansvar mellem klasser, genbrug af kode samt strukturering af assets og **Blueprints** i projektet. For at komme i gang med projektet oprettede gruppen først selve Unreal Engine-projektet. Projektet blev initialt oprettet som et C++-projekt baseret på Unreal Engine's **FirstPersonTemplate**, da gruppen ønskede, at spillet skulle opleves fra et førstepersonsperspektiv.

Dette valg blev truffet tidligt i processen for at sikre, at gameplay-oplevelsen blev defineret allerede fra starten, hvilket senere fik betydning for designvalg og udviklingen af spilmekanikker.

Efter oprettelsen af projektet blev der opsat et fælles GitHub-repository for at understøtte samarbejdet i gruppen. Dette gjorde det muligt at arbejde parallelt på projektet, samtidig med at der blev skabt en struktureret versionstyring af filer og ændringer.

Brugen af pull requests og code reviews blev implementeret for at sikre kvalitet i koden og for at give mulighed for faglig sparring mellem gruppemedlemmerne. Denne arbejdsform viste sig at være vigtig for at undgå konflikter i koden og for at sikre, at ændringer blev gennemgået og forstået af flere i gruppen, før de blev integreret i hovedprojektet.

Tidligt i udviklingsfasen blev der eksperimenteret med Unreal Engine, særligt i forhold til samspillet mellem **Blueprints** og C++. Gruppen undersøgte blandt andet, hvordan **Blueprints** kunne anvendes til hurtig prototyping af gameplay-elementer, mens C++ blev brugt til mere komplekse og performancekritiske systemer.

Denne kombination viste sig at være central for projektets udvikling, da den gav en balance mellem fleksibilitet og kontrol. Eksempelvis blev gameplay-relaterede funktioner ofte udviklet som **Blueprint**-baserede actors og widgets, mens centrale systemer som netværkslogik, sessionshåndtering og spillerdata blev implementeret i C++-klasser såsom **GameMode**, **GameState**, **PlayerController** og **GameInstance**.

I begyndelsen af projektet bar arbejdet præg af eksperimentering og læring, hvor gruppen brugte tid på at opbygge forståelse for engine'ens struktur og arbejdsmetoder. Dette betød, at fremdriften i starten var relativt langsom, men til gengæld lagde det

et solidt fundament for den videre udvikling.

Efterhånden som gruppen fik større teknisk forståelse, blev arbejdsprocessen mere effektiv, og udviklingen af spillets funktioner kunne ske mere struktureret og målrettet. Gruppen begyndte her at arbejde mere systematisk med opdeling af ansvar mellem klasser, genbrug af kode samt strukturering af assets og **Blueprints** i projektet.

4.2 Game Sessions

En af de første centrale dele af systemet, som gruppen udviklede, var online sessions-systemet. Dette blev prioriteret tidligt i projektforsløbet for at sikre, at alle efterfølgende systemer kunne testes og udvikles i en reel multiplayer-kontekst. Samtidig gav det gruppen mulighed for tidligt at etablere en stabil teknisk base, som resten af spillets netværksfunktionalitet kunne bygges ovenpå.

Systemet blev udviklet ved hjælp af Unreal Engine's `OnlineSubsystem`, som tilbyder en række værktøjer til håndtering af multiplayer-funktioner. Implementeringen omfattede blandt andet oprettelse og håndtering af sessions, grundlæggende match-making samt netværkskommunikation mellem klienter og servere.

Dette udgjorde en vigtig milepæl i projektet, da det var første gang gruppen fik etableret fungerende multiplayer-infrastruktur, hvilket senere blev afgørende for hele spillets struktur og funktionalitet.

I den første version af systemet blev der implementeret en relativt simpel løsning, hvor en spiller kunne oprette en session, og andre spillere automatisk blev koblet til den først oprettede session. Denne funktionalitet blev implementeret gennem funktioner i `GameInstance`-klassen, som håndterede oprettelse, søgning og tilslutning til sessions via Unreal Engine's online subsystem.

Denne løsning fungerede som et udgangspunkt for at få systemet teknisk stabilt, men viste sig senere at være begrænset i forhold til brugeroplevelse og fleksibilitet.

Senere i udviklingsforsløbet blev systemet derfor videreudviklet til den nuværende løsning (jf. bilag F.2.29–F.2.30), hvor spillere i stedet kunne vælge mellem tilgængelige sessioner. Der blev desuden implementeret en lobby-funktion, hvor spillere kunne samles og vente, inden spillet startede.

Denne ændring blev foretaget på baggrund af test og overvejelser om brugeroplevelse, da gruppen vurderede, at det var vigtigt at give spillerne mere kontrol og bedre overblik over multiplayer-delen af spillet.

Sessionhåndteringen blev implementeret i Unreal Engine's `GameInstance`-klasse, som er designet til at bevare data og funktionalitet på tværs af levels og spilstilstande. Ved at placere al sessionlogik i `GameInstance` sikrede gruppen, at sessiondata kunne bevares uanset, hvilken del af spillet spilleren befandt sig i.

Dette var en vigtig teknisk beslutning, da det bidrog til en mere stabil og konsistent

multiplayer-oplevelse og reducerede risikoen for datatab ved level-skift.

For at gøre sessionssystemet tilgængeligt for spilleren blev der desuden udviklet et brugerinterface, **SessionBrowserUI**, hvor spilleren kunne se en liste over aktive sessioner og vælge, hvilken session vedkommende ønskede at deltage i.

Dette UI blev implementeret ved hjælp af Unreal Engine's Widget System, som gør det muligt at opbygge interaktive og dynamiske brugergrænseflader. UI'et inkluderede funktioner til realtidsopdatering af sessionlisten, visning af relevante sessiondetaljer samt håndtering af brugerinput til både oprettelse og tilslutning af sessions.

Forbindelsen mellem UI'et og sessionlogikken blev håndteret gennem **GameInstance**-klassen, som fungerede som bindeled mellem brugerens input og Unreal Engine's on-line subsystem.

4.3 TeamSelection

Dette system blev designet for at give spillerne mulighed for at vælge mellem forskellige hold, hvilket var en central del af spillets multiplayer-struktur og gameplay-dynamik.

Implementeringen af Team Selection blev i første omgang lavet ved hjælp af Unreal Engine's tag-system. Dette blev valgt som en enkel og fleksibel løsning, da alle spillere i den tidlige version af spillet havde samme basiskarakter, før et egentligt klassesystem blev implementeret.

Gruppen udviklede i denne forbindelse et widget-baseret system, hvor spilleren fik tildelt et specifikt tag baseret på deres holdvalg. Dette blev håndteret gennem `TeamSelectionUI`, som registrerede spillerens valg og sendte informationen videre til `GameMode`-klassen.

For at understøtte dette blev der oprettet spawn points i scenen med tilsvarende tags, således at systemet kunne matche spillerens tag med det relevante spawn point. På denne måde blev spilleren automatisk spawn'et på den korrekte placering afhængigt af det valgte hold (jf. bilag F.2.3). Denne løsning gjorde det muligt at skabe en simpel, men effektiv logik til holdopdeling i den tidlige fase af udviklingen.

Som projektet udviklede sig, blev der senere implementeret et mere avanceret nedarvningssystem for spillerklasser, som gjorde det muligt at differentiere spillernes egenskaber og roller mere detaljeret. Dette klassesystem vil blive behandlet senere i rapporten.

`TeamSelectionSystem` blev implementeret i Unreal Engine's `GameMode`-klasse, da denne klasse håndterer spillets overordnede regler og logik, herunder spawning af spillere og styring af holdfordeling. Ved at placere logikken her sikrede gruppen, at systemet fungerede globalt på tværs af levels og spiltilstande, hvilket var vigtigt for at opretholde en konsistent multiplayer-oplevelse.

For at gøre systemet tilgængeligt for spilleren blev der udviklet et UI, hvor spilleren kunne vælge mellem de tilgængelige hold. Dette UI blev implementeret via Unreal Engine's Widget System, som gjorde det muligt at skabe en interaktiv og brugervenlig grænseflade. UI'et indeholdt blandt andet visning af holdnavne og tilhørende knapper, som hjalp spilleren med at træffe et valg.

UI'et blev instansieret gennem `PlayerController`-klassen, da denne klasse hånd-

terer spillerens input og interaktion med spillet. Denne placering gjorde det muligt at sikre en effektiv kobling mellem brugerinput og spillets logik. Når spilleren valgte et hold, blev informationen sendt videre til **GameMode**, som herefter stod for spawning af spilleren på det korrekte spawn point.

Systemet blev løbende testet og itereret gennem udviklingsprocessen for at sikre stabil funktionalitet og en god brugeroplevelse, da holdvalg var en vigtig del af spillets multiplayer-dynamik (jf. bilag F.2.10–F.2.11).

Efter at spilleren havde valgt et hold, blev vedkommende sendt videre til næste UI, hvor spilleren kunne vælge deres klasse (jf. bilag F.2.33). Denne opdeling i flere trin bidrog til en mere struktureret onboarding-proces i spillet og gjorde det lettere for spilleren at forstå deres rolle i spillets startfase.

4.4 VoiceChat

//tænker Victor dit kan være her

4.5 Timer

//tænker Jonathan dit kan være her

4.6 Class Selection

Som tidligere nævnt har de forskellige hold også tilknyttede klasser, som spillerne kan vælge imellem. Dette system blev implementeret for at give spillerne mulighed for at differentiere sig yderligere inden for deres valgte hold og for at tilføje mere dybde til gameplayet, da hver klasse kunne have unikke evner og roller i spillet.

For at udvikle klassesystemet blev der taget udgangspunkt i en fælles base-klasse, **PlayerCharacter**, hvor alle fælles funktioner og egenskaber blev defineret. Herefter blev der oprettet specifikke klasser for hvert hold. Disse klasser indeholdt fælles hold-specifikke egenskaber, hvor eksempelvis **ResistanceTeam** kunne samle nøgler op, mens **NaziTeam** havde evnen til at fange modstandere. Disse mekanikker blev designet som centrale gameplay-elementer og vil blive uddybet senere i rapporten.

Denne nedarvningsstruktur gjorde det muligt at organisere koden på en måde, der understøttede genbrug og modularitet, samtidig med at der var plads til specialisering af de enkelte klasser. Gruppen oplevede, at denne tilgang gjorde systemet mere overskueligt at arbejde med, særligt i takt med at antallet af klasser voksede.

Derfra blev der implementeret yderligere nedarvede klasser inden for hvert hold, hvor hver enkelt klasse fik tildelt unikke evner og egenskaber. Dette gjorde det muligt at skabe en mere varieret og strategisk gameplay-oplevelse, hvor spilleren kunne vælge en rolle, der passede til deres spillestil.

For **ResistanceTeam** blev der eksempelvis udviklet fire forskellige karakterklasser: **SportClass**, **ScienceClass**, **MusicClass** og **LanguageClass**. Tilsvarende blev **NaziTeam** udvidet med to karakterklasser: **GestapoSoldier** og **SSOfficer**. Hver klasse blev desuden udstyret med unikke meshes, hvilket gjorde det muligt visuelt at skelne mellem både klasser og hold i spillet.

I praksis foregik klassevalget ved, at spilleren først blev sendt videre fra **TeamSelectionUI** til et klassevalg-UI (jf. bilag F.2.33). Herfra blev spilleren dirigeret videre til Unreal Engine-baserede UI'er for henholdsvis **ResistanceUI** (jf. bilag F.2.28) og **NaziUI** (jf. bilag F.2.27).

Disse UI'er bestod af interaktive knapper, som kaldte en funktion, der spawner spillerens karakter, tildeler det korrekte team-tag og placerer spilleren på det relevante spawn point. Dette blev håndteret gennem funktioner i henholdsvis **GameMode**- og **PlayerController**-klasserne, som stod for spawning og initialisering af spilleren.

Når karakteren var blevet spawnet korrekt, kunne spilleren vælge deres specifikke klasse inden for det valgte hold til den pågældende spil-session. Denne opdeling i flere trin gjorde det muligt at strukturere onboarding-processen i spillet og sikre, at spilleren gradvist blev introduceret til deres rolle og muligheder i spillet.

4.7 Speceille Evner

//tænker Jonathan dit kan være her

4.8 nøgler og kisten

For at løse use casene omkring at finde nøgler og låse en kiste op, som fungerer som en reference til skolens symbolik og historie, blev der udviklet et dedikeret gameplay-system. Systemet blev implementeret ved at oprette en Actor kaldet **BPA_Key** i spillet. Denne Actor indeholder en collider, som registrerer, når en spiller interagerer med objektet. Når en spiller kommer i kontakt med collidieren, udløses en event, som sender en besked til spillets **GameState** om, at spilleren har samlet en nøgle.

Implementeringen er baseret på Unreal Engine's event-system, hvor en overlap-event på collidieren anvendes til at detektere spillerens interaktion med nøglen. Når denne event trigges, sendes der en besked videre til **GameState**-klassen, som håndterer den overordnede logik for registrering af nøgler.

Denne løsning blev valgt, fordi **GameState** er en replicated klasse, hvilket betyder, at den kan synkronisere information på tværs af alle klienter i et multiplayer-spil (jf. bilag F.2.16 og F.2.7). Når en spiller har samlet en nøgle, broadcaster **GameState** desuden en opdatering til alle klienter, så deres UI bliver opdateret og viser, at en nøgle er blevet fundet (jf. bilag F.2.18). Dette bidrager til en mere dynamisk og fælles spiloplevelse, hvor alle spillere løbende kan følge med i spillets progression.

For at gøre placeringen af nøglerne mere dynamisk blev der udviklet et separat script til en Actor kaldet **BPA_KeySpawnArea**. Denne Actor fungerer som et afgrænset område med en bounding box (jf. bilag F.2.17), hvorfra der kan genereres tilfældige spawn-positioner. Ved spilstart beregner systemet et tilfældigt punkt inden for dette område, hvor nøglen derefter spawnes.

Denne løsning blev valgt for at øge variationen i gameplayet, da nøglerne dermed ikke altid er placeret samme sted i hver spil-session. For at sikre korrekt synkronisering på tværs af alle klienter blev spawn-logikken implementeret i **GameMode**-klassen (jf. bilag F.2.4), som håndterer spillets overordnede regler og initiering af spilobjekter.

Efterfølgende blev der udviklet et tilsvarende system for kisten (chest). Kisten fungerer på mange måder på samme måde som nøglen og indeholder også en collider, der registrerer spillerinteraktion. Her kontrollerer systemet dog, om spilleren tilhører Resistance team, og hvis dette er tilfældet, aktiveres spillets win condition (jf. bilag F.2.14), som vil blive uddybet senere i rapporten.

Ligesom med nøglerne blev kistens placering også koblet til key spawn-området,

så den ikke altid befinder sig samme sted i hver spil-session. Dette blev ligeledes håndteret via **GameMode** for at sikre synkronisering mellem klienter og konsistent gameplay (jf. bilag F.2.4).

Samlet set skaber dette system en gameplay-mekanisme, hvor spillerne skal udforske kortet, finde nøgler og til sidst låse kisten op for at vinde spillet. Dette bidrager til en dynamisk spiloplevelse, hvor både samarbejde og strategisk bevægelse på kortet bliver centrale elementer.

4.9 Fange Mechanic

For at implementere mekanikken omkring at fange modstandere, som er en central del af Nazi team-gameplayet, blev der tilføjet en collider på parent-klassen for alle Resistance team-spillere (bilag F.2.13). Denne collider kommunikerer via en variabel, som er tilgængelig for Nazi team-spilleren.

Når der opstår et overlap, og variabelen er aktiv, trigges der logik på Nazi team-spilleren (bilag F.2.9). Denne logik er implementeret i et Blueprint, som kalder et custom C++-funktionalitet, der er defineret i klassen **Sep4Character** (bilag F.1.7 og F.1.8). Denne klasse fungerer som den overordnede karakterklasse, som alle spillets characters nedarver fra.

Funktionaliteten håndterer, at den fangede spiller “fastholdes” og derefter via serveren får besked om at blive teleporteret til en fast position i banen. Dette sikrer, at handlingen bliver synkroniseret korrekt i multiplayer-miljøet og dermed er ens for alle spillere.

Det område, spilleren bliver teleportet til, er implementeret som en separat Actor kaldet **BPA_JailArea**. Denne Actor består af fire vægge, som afgrænser området og definerer “fængslet” i spillet.

Derudover er der implementeret en dør som en selvstændig Actor, **BPA_Door**, som indeholder logik til både åbning og lukning. Døren lukkes automatisk ved spillets start via det custom **GameMode** (bilag F.2.6).

Døren indeholder desuden en collider, som gør det muligt for andre spillere at interagere med den for at befri en fanget spiller. Denne befrielsesmekanik vil blive uddybet senere i rapporten.

4.10 befri fængslet

Som tidligere nævnt er der implementeret en mekanik, hvor spillere på Resistance-teamet kan befri fangede holdkammerater fra fængslet. Denne funktionalitet blev implementeret ved at tilføje en collider til døren i `BPA_Door`-klassen, som registrerer, når en spiller interagerer med objektet.

Når en spiller kommer i kontakt med collideren, udløses en event, som sender en besked til serveren om, at døren skal åbnes. Serveren håndterer herefter den overordnede logik for åbning af døren (bilag F.2.15). Denne serverbaserede tilgang sikrer, at handlingen bliver synkroniseret korrekt i multiplayer-spillet, så alle spillere oplever den samme ændring i spilverdenen samtidig.

4.11 Win Conditions

Da der allerede var implementeret en timer i spillet, og systemet med nøgler og kiste var på plads, var det en relativt overskuelig opgave at implementere et win condition-system, som samler disse mekanikker. Dette blev gjort ved at tilføje en funktion i `GameState`-klassen, som løbende evaluerer, om enten timeren er udløbet, eller om kisten er blevet åbnet (jf. bilag F.2.8).

Når en af disse betingelser opfyldes, broadcaster systemet en besked til alle klienter, som aktiverer et slutspils-UI, hvor spillerne kan se, om deres hold har vundet eller tabt (jf. bilag F.2.19 og F.2.20). Samtidig stoppes den løbende spilopdatering (Game Tick) for at sikre, at spillets tilstand fryses ved afslutning, så resultatet ikke ændres efter win condition er opnået.

Denne løsning sikrer en klar og synkron afslutning på spillet for alle spillere og binder de forskellige gameplay-systemer sammen til én samlet spiloplevelse.

4.12 map opsætning

For at udvikle mappet til spillet så realistisk som muligt, blev der anvendt specialiseret hardware i form af kameraer til optagelse af 360-graders video af relevante lokationer på Viborg Katedralskole. Disse videoer blev efterfølgende opdelt i stillbilleder, som blev brugt som datagrundlag til photogrammetry-processen.

Til dette blev RealityScan anvendt, som er et værktøj udviklet af Epic Games. Softwaren analyserer en stor mængde billeder, hvor hvert billede indeholder information om perspektiv, position og vinkel. På baggrund af dette genererer programmet en point cloud, som efterfølgende anvendes til at opbygge et 3D mesh samt tilhørende materialer. Denne proces gør det muligt at skabe meget realistiske 3D-modeller og blev derfor valgt som en central metode til opbygning af spillets miljø.

Metoden viste sig dog også at have udfordringer i praksis. Gruppen erfarede, at optagelserne krævede høj præcision og en vis teknisk erfaring for at opnå optimale resultater. Dette medførte, at nogle af de genererede modeller fik “interessante” eller urealistiske udtryk (jf. bilag E). På trods af dette var mange af lokationerne stadig genkendelige og anvendelige i spillet, mens andre måtte forkastes helt. Denne begrænsning vurderes som en naturlig konsekvens af, at området er væsentligt større end det, værktøjet typisk er designet til at håndtere.

En anden udfordring opstod i forhold til den tekniske implementering af de genererede meshes i spillet. Da standard-output fra photogrammetry primært indeholder simple collision-boxes, har gruppen været nødt til manuelt at tilføje colliders til de enkelte meshes for at sikre korrekt spillerinteraktion. Dette har været en tidskrævende proces, men nødvendig for at sikre, at spilleren ikke kunne bevæge sig igennem faste objekter i miljøet.

Derudover har sammensætningen af de enkelte meshes til et sammenhængende map også krævet manuelt arbejde. Modellerne blev placeret enkeltvis og justeret, så spilleren kunne bevæge sig mellem områderne uden tydelige overgange. Dette har dog medført visse udfordringer i forhold til spilbarhed og navigation, da det i nogle tilfælde krævede kreative løsninger for at guide spilleren og forhindre, at de faldt uden for kortets grænser.

En yderligere problemstilling ved denne tilgang er, at RealityScan-genererede meshes ofte indeholder “mellemområder”, som udfylder huller i point cloud-data. Disse

områder er ikke baseret direkte på optagelser, hvilket resulterer i uregelmæssige og ofte abstrakte former. Dette har skabt udfordringer i forhold til collision og spillerbevægelse, da spilleren i nogle tilfælde kunne bevæge sig ind i disse områder eller risikere at falde igennem geometrien.

Set i retrospektiv vurderer gruppen, at et portalsystem, hvor spilleren teleporteres mellem separate områder, muligvis havde været en bedre løsning. Dette kunne have reduceret problemer med navigation og collision samt givet en mere kontrolleret og visuelt konsistent oplevelse. Samtidig kunne det have forbedret den samlede spiloplevelse.

4.13 AI NPC'er

På trods af at spillet primært er udviklet som et multiplayer-spil, var idéen om også at understøtte singleplayer en vigtig prioritet i designfasen. For at realisere dette blev AI-NPC'er implementeret som en løsning, der kunne simulere modstandere og medspillere i spillet.

Til at starte med blev der oprettet en AI-nedarvning af alle karakterklasser. Derudover fik host-spilleren et dedikeret UI, hvorfra det var muligt at spawn enten tilfældige Resistance team-klasser eller tilfældige Nazi team-karakterer (bilag F.2.22). Dette gjorde det muligt at teste og spille spillet uden andre spillere online, samtidig med at de samme gameplay-systemer kunne genbruges.

På alle AI-nedarvede karakterer blev der tilføjet et AI Sense-komponent, som giver AI'en mulighed for at "se" og registrere objekter i spillets verden. Dette blev brugt til at give NPC'erne perception og reaktionsevne, så de kunne interagere med spillets elementer på en mere dynamisk måde.

Derudover blev der implementeret logik, som styrer AI'ens adfærd afhængigt af dens rolle. Hvis en Resistance team-AI opdager en nøgle, vil den prioritere at bevæge sig mod og samle den i stedet for at roame tilfældigt. Omvendt vil en Nazi team-AI aktivt forsøge at opspore og forfølge Resistance team-spillere.

Denne adfærd blev implementeret i en custom AIController (bilag F.2.1), som håndterer beslutningslogik og styring af NPC'ernes handlinger. AIControlleren fungerer som hjernen i systemet og sikrer, at NPC'erne reagerer passende på deres omgivelser og spillets tilstand.

Samlet set gjorde denne løsning det muligt at understøtte singleplayer-oplevelsen uden at ændre grundlæggende på spillets multiplayer-arkitektur, da AI'en er bygget oven på de samme karakter- og gameplay-systemer.

ud over dette, gjorde det også sådan at flere spiller, ville kunne spille store spil, med mange modstander, da AI'en også blev synkroniserede ud til alle spiller.

4.14 afsluttende forbedringer

for at gøre spillet til sidst mere en samlede færdig oplevelse, blev der først lavet mulighed for Session hosten, for at

5 Personlige Refleksioner

5.1 Jonathan

Jeg tog en mindre rolle i projektets opstart og dette påvirkede mit engagement i de tidlige faser af projektet, og jeg oplevede også perioder, hvor jeg ikke deltog i møder eller informerede gruppen om fravær på forhånd. Set i bakspejlet kunne jeg have bidraget mere aktivt. Jeg tror, at min manglende deltagelse i starten kan have været en form for social loafing, hvor jeg ubevidst trak mig tilbage fra ansvaret, fordi jeg følte, at andre i gruppen ville tage sig af opgaverne, eller allerede havde taget sig af dem. For at bryde ud af denne dynamik forsøgte jeg at bidrage på områder, hvor jeg oplevede, at der manglede arbejdskraft, blandt andet gennem kildearbejde og rapportskrivning. Da jeg som person har en tendens til at være tilbageholdene i sociale sammenhænge, var dette en måde jeg stadig kunne bidrage på. Jeg oplevede at gruppen håndterede situationen professionelt og inkluderende, hvilket gjorde det lettere for mig at bidrage mere aktivt senere i projektforsløbet. Jeg har lært, at det er vigtigt at kommunikere åbent om ens deltagelse og forpligtelser i et gruppeprojekt for at sikre, at alle medlemmer føler sig ansvarlige og engagerede. I fremtidige projekter vil jeg stræbe efter at være mere proaktiv i min deltagelse og kommunikere klart med mine gruppemedlemmer om mine bidrag og eventuelle udfordringer, jeg måtte have.

5.2 Victor

5.3 Nikolaj

6 Refleksion over vejledning

Gruppen har oplevet, at vejledningen har fungeret godt i den del af forløbet, hvor der stadig har været planlagt undervisning og faste vejledningstimer. I denne periode har det været muligt at få støtte og sparring, hvilket har bidraget positivt til projektets fremdrift.

Samtidig har gruppen oplevet, at det kan være begrænsende kun at have en enkelt vejleder tilknyttet projektet, da vejlederen ikke nødvendigvis har haft faglig indsigt inden for alle relevante områder. Dette har i nogle tilfælde gjort det vanskeligt at få helt konkret og brugbar feedback på mere specialiserede problemstillinger, særligt i forhold til de Formaila aspekter af projektet.

Efter den planlagte undervisnings- og vejledningsperiode har gruppen desuden erfaret, at det kræver en mere proaktiv tilgang selv at tage kontakt til vejlederne ved opståede problemer eller tvivlsspørgsmål. Gruppen har i den forbindelse lært, at det er vigtigt aktivt at opsøge vejledning løbende, frem for at afvente faste møder.

Fremadrettet vil gruppen derfor fokusere på at være mere opsøgende og struktureret i brugen af vejledere, blandt andet ved at forberede konkrete spørgsmål på forhånd og tage kontakt tidligere i processen, når udfordringer opstår. Dette forventes at kunne forbedre udbyttet af vejledningen og bidrage til en mere effektiv projektudvikling.

7 Konklusion